FDX Consensus Standard Data Format (v1) API Security Model

Version 5.0 May 2025

FDX Security and Authentication Working Group

Legal Notice

Financial Data Exchange, Inc (FDX®) is a standards body and adopts this API Security Model document for general use among industry stakeholders. Many of the terms, however, are subject to additional interpretations under prevailing laws, industry norms, and/or governmental regulations. While referencing certain laws that may be applicable, readers, users, members, or any other parties should seek legal advice of counsel relating to their particular practices and applicable laws in the jurisdictions where they do business. See FDX's complete Legal Disclaimer located at http://www.financialdataexchange.org for other applicable disclaimers. Copyright © 2025 - FDX - All Rights Reserved.

Revision History

Document Version	Notes	Date
5.0	Updated for RFC 0353 Consensus Standard Data Format Security Model and 0354 Consensus Standard Data Format Security Profile. Removed previous document content, removed references to sub-intermediaries per RFC 0359.	**May 2025**
4.2	Minor formatting updates	**November 2024**
4.1	RFC 0237 FDX API Security Profile 1.0 has been included in Part 1 instead of referenced. RFC 0289 non-material document updates have been incorporated.	**September 2024**
4.0	RFC 0011 has been incorporated as Part 4 End to End Encryption. This was formerly published as section 4 of the FDX API Specification document though its version 6.0.	**December 2023**
3.5	Updates for recipient registration guidelines, automation, and delegation – Part 2.	**May 2022**
3.4	Updates for FAPI Security Profile 1.0 – Part 1 and Part 2.	**October 2021**
3.3	Includes FDX RFC 0117 FDX OAuth Extensions for identifying intermediary entities and FDX RFC 0115 Guidelines and Best Practices for Sharing Sensitive Data Securely.	**May 2021**
3.2	Initial Document Release	**December 2020**

Contents

Legal Notice 2
Revision History 2
Contents 3

CHANGE LOG 4

FOREWORD 7

INTRODUCTION 7

Notational Conventions 7
Scope 7
Note on Normative References 8
Terms and Definitions 8
Symbols and Abbreviations 8
Updates to other FDX RFCs and Conflicts 8
Constituent Standards and Conformance to the Security Model 9

PART 1: FDX CONSENSUS STANDARD DATA FORMAT V1 API SECURITY PROFILE 10

PART 1A: FDX CONSENSUS STANDARD DATA FORMAT V1 API SECURITY PROFILE - GREEN 10

1.1 Overview 10
1.2 Profile 10

PART 1B: FDX CONSENSUS STANDARD DATA FORMAT V1 API SECURITY PROFILE - BLUE 13

1.3 Overview 13
1.4 Profile 13

PART 2: FDX MESSAGE ENCRYPTION 15

2.1 Overview 15
2.2 Message Encryption 15
2.3 Key Management 15
2.4 Examples 17

PART 3: FDX STEP-UP AUTHENTICATION 21

3.1 Overview 21
3.2 OAuth 2.0 Step-up Authentication Challenge Protocol 21
3.3 Implementer's Notes 21

NORMATIVE REFERENCES 23

INFORMATIVE REFERENCES 23

Change Log

Document Version	Date	Originator/ Contributor	Reason for Change	Ratified
5.0	May 2025	Authors: Ray Voss Mark Andrus George Fletcher Lukasz Jaromin Contributors: Sudipta Sarkar Vernon Miller Erik Umenhofer Ron Dodge Stephen Halm Stephen Ipe Varghese Anil Mahalaha Aravind Nuthalapati	RFC 0353 - FDX Consensus Standard Data Format v1 API Security Model RFC 0354 - FDX Consensus Standard Data Format v1 API Security Profile 1.0	Yes
4.2	November 2024	Clyde Cutting	RFC 0289 - FDX API Security Model V4.0 - non-material document updates	Yes
4.1	September 2024	Ray Voss Lukasz Jaromin Venu Mandulapalli	RFC 0237 - FDX API Security Profile 1.0	Yes
4.0	December 2023	Ravneet Singh Ray Voss Jeff Johnson	RFC 0288 - FDX API Specification V6.0 document errata A section on end-to-end encryption from the API Specification document has been moved into this Security Model document.	Yes

Document Version	Date	Originator/ Contributor	Reason for Change	Ratified
3.5.1	May 2025	Clyde Cutting	RFC 0359 Errata to Remove References to Sub-Intermediaries	Yes
3.5	May 2022	Jeff Schulte Lukasz Jaromin Dinesh Katyal	RFC 0153 - Recipient Registration Automation	Yes
		Ben White Ray Voss Michael Cypher Max Rozen Dinesh Katyal	RFC 0206 - Recipient Registration With Delegation to an Ecosystem Registry	
3.4	October 2021	Dinesh Katyal Wesley Dunnington	RFC 0122 - FDX Adoption of FAPI	Yes
		Clyde Cutting Bruce Wilcox Paul Allen Rick Hollingsworth Ray Voss Ben Simmons Aaron Parecki Wes Dunnington	RFC 0135 - FDX Tax security using Basic Authorization	
3.3	May 2021	Shawn Jobe Dinesh Katyal	RFC 0115 - Guidelines and Best Practices for Sharing Sensitive Data Securely	Yes
		Anil Mahalaha Aaron Parecki Dinesh Katyal	RFC 0117 - FDX OAuth Extensions	

Document Version	Date	Originator/ Contributor	Reason for Change	Ratified
3.2	December 2020		Initial Document Release	

Foreword

Financial Data Exchange, Inc. (FDX) is a standard-setting organization operating in the United States and Canada. It has over 200 member organizations, including depository and non-depository commercial entities, Data Providers, Third Parties, Data Aggregators, service providers to open banking participants, trade and industry organizations and other non-commercial members, including consumer groups. FDX's primary purpose is to develop, improve and maintain a common, interoperable standard for secure consumer and business access to financial information.

In January 2025, the Consumer Financial Protection Bureau (CFPB) issued an order recognizing FDX as a standard setting body (SSB) under the CFPB's Required Rulemaking on Personal Financial Data Rights ("the rule") [RR PFDR]. As an SSB, FDX is authorized to issue consensus standards that promote high standards of security and interoperability among implementers and that covered entities may use to help them comply with certain requirements in the rule.

Introduction

The requirements and protocols for communicating requests and responses for FDX Consensus Standard Data Format v1 API endpoints must be defined in order to: (i) ensure interoperability among implementers; (ii) promote protection of confidentiality, integrity, and availability of consumer financial data; and (iii) and to lower costs of implementation. This RFC specifies the ***FDX Consensus Standard Data Format (v1) API Security Model v5.0*** ("the security model") that defines these requirements and protocols. FDX will continue to assess protocols and requirements for communicating requests and responses as appropriate for this security model, including tokenization, registration and consent, among other security domains and controls.

Notational Conventions

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document **SHALL** only be interpreted as described in IETF RFC 2119. These key words **SHALL NOT** be interpreted with their natural language meanings.

Scope

This security model specifies requirements and protocols for communicating requests and responses for data as defined in *Financial Data Exchange Consensus Standard Data Format, version 1 (CSDF v1)*.

Note on Normative References

For all dated references only the version cited applies. Previously and subsequently published versions of all dated references are not applicable. Normative references are listed below.

Terms and Definitions

The terms defined in [RR PDFR], [IETF RFC 6749] apply to this security model.

Token caching strategy: Token caching strategy refers to an implementer's approach in managing existing access tokens after the implementer performs a step-up authentication challenge and creates a new token with sufficient recentness and authentication level for the requested resource in the given context. [IETF RFC 9470]

Symbols and Abbreviations

CFPB - Consumer Financial Protection Bureau

CSDF - Consensus Standard Data Format

FDX - Financial Data Exchange

RR PFDR - Required Rulemaking on Personal Financial Data Rights

SSB - Standards Setting Body

Updates to other FDX RFCs and Conflicts

This specification replaces and makes obsolete the following FDX standards listed in Table 1.

Table 1: FDX Standards Replaced and Made Obsolete

FDX RFC	Title
N/A	All previous FDX API security models
FDX RFC 0011	Message Encryption
FDX RFC 0122	FDX Adoption of FAPI
FDX RFC 0218	Control Considerations for Step Up Authentication
FDX RFC 0223	FDX Adoption of FAPI 1.0 Part 1: Baseline
FDX RFC 0237	FDX API Security Profile 1.0

Constituent Standards and Conformance to the Security Model

The FDX Consensus Standard Data Format v1 API Security Model is comprised of its constituent standards. These include:

- FDX Consensus Standard Data Format v1 API Security Profile - Green
- FDX Consensus Standard Data Format v1 API Security Profile - Blue
- FDX Message Encryption as specified in this standard
- FDX Step-up Authentication as specified in this standard

In order to conform to the FDX Consensus Standard Data Format v1 requirements and protocols for communicating requests and responses, implementations **MUST** conform to *FDX Consensus Standard Data Format v1 API Security Profile - Green* **OR** *FDX Consensus Standard Data Format v1 API Security Profile - Blue*.

FDX Message Encryption and *FDX Step-up Authentication* are conditional requirements for conformance with the FDX Consensus Standard Data Format v1.

Agreements between implementers **SHALL** govern whether *FDX Consensus Standard Data Format v1 API Security Profile - Green* or *FDX Consensus Standard Data Format v1 API Security Profile - Blue* applies to their implementations of the FDX Consensus Standard Data Format v1.

Agreements between implementers **SHALL** govern whether *FDX Message Encryption* or *FDX Step-up Authentication* **SHALL** apply to their implementations of the FDX Consensus Standard Data Format v1. If agreements between implementers specify application-level encryption or step-up authentication, then the respective standards apply.

Part 1: FDX Consensus Standard Data Format v1 API Security Profile

Part 1A: FDX Consensus Standard Data Format v1 API Security Profile - Green

1.1 Overview

1.1.1 Introduction

The *FDX Consensus Standard Data Format v1 API Security Profile - Green* specifies requirements and protocols of the FDX Consensus Standard Data Format v1 for communicating requests and responses. This profile is based on OAuth 2.0 and includes additional requirements as noted.

FDX CSDF v1 API Security Profile - Green is designed to suit the business needs of implementers for whom the *FDX CSDF v1 API Security Profile - Blue* is not warranted. Green is based on OAuth 2.0 [IETF RFC 6749] and includes the constraints described here.

1.1.2 Applicability

Agreements between implementers **SHALL** govern whether *FDX Consensus Standard Data Format v1 API Security Profile - Green* **SHALL** apply to their implementation of the FDX Consensus Standard Data Format v1.

1.1.3 Scope

The FDX Consensus Standard Data Format v1 API Security Model is intended to enable covered entities as defined in *RR PFDR* to meet the security and interoperability objectives of the rule by defining standard requirements and protocols for communicating requests and responses.

1.2 Profile

1.2.1 Authorization code flow

OAuth 2.0 with authorization code flow **SHALL** be used for the following FDX Consensus Standard Data Format v1 Endpoints:

- None

1.2.2 Client credentials grant

OAuth 2.0 with client credentials grant **SHALL** be used for the following FDX Consensus Standard Data Format v1 Endpoints:

- Core
- Customer
- Money Movement

- Notifications - Get
- Notifications - Subscription
- Meta

1.2.3 Other endpoints

Implementers **SHALL**, for client authentication, use client certificate authentication TLS per [IETF RFC 5246] **OR** private_key_jwt per [IETF RFC 7521] and [IETF RFC 7523] for the following Consensus Standard Data Format v1 Endpoints:

- Notifications - Publishing

1.2.4 Additional requirements

1. Implementers **SHALL** use confidential clients only.
2. Authorization servers **SHALL** verify whether scopes included in the authorization request match the consent request object. If there are any extra scopes included in the authorization request, then the request **SHALL** fail.
3. For each request the resource server **SHALL** verify the consent record associated with the access_token entitles the requestor to perform the action on the target resource. If there are any extra scopes or actions, then the request **SHALL** fail.
4. Access tokens **SHALL** expire after no more than 900 seconds.
5. Suppress Referer Headers:
    a. The client **SHALL** use Referrer Policy to control the information sent in the Referer header. (NOTE: For example, using `Referrer-Policy: no-referrer` prevents the Referer header from being sent entirely, while `Referrer-Policy: same-origin` only sends the header for same-origin requests.)
6. Secure Token Transmission:
    a. Tokens **SHALL** be transmitted securely using HTTP headers rather than being included in URLs.(NOTE: for example, in the `Authorization` header.)
7. Issuer Identification:
    a. The client **SHALL** verify the issuer of the authorization response. (NOTE: This can be done by checking the `iss` (issuer) claim in tokens or responses to confirm that they originate from the expected authorization server.)
8. Distinct Redirect URIs:
    a. The client **SHALL** use distinct redirect URIs for different authorization servers. (NOTE: This helps the client identify which server the response is coming from and prevents confusion between responses from different servers.)
    b. Nonce Parameter (Note: for OpenID Connect). If using OpenID Connect, the client **SHALL**:
        i. Employ the `nonce` parameter to protect against replay attacks
        ii. Ensure that the response is tied to the specific request initiated by the client.
9. Refresh token Rotation:

a. The authorization server **SHALL** comply with section 4.14 of [IETF RFC 9700] pertaining to refresh token rotation.

10 Audience Restriction:

a. The authorization server **SHALL** ensure that access tokens are audience-restricted, meaning they are valid only for specific resource servers, via an audience claim in the token that specifies the intended recipient.

11 Scope Restriction:

a. The authorization server **SHALL** limit the scope of access tokens to the minimum necessary for the client's operations.

Table 2: Consensus Standard Data Format v1 Endpoint Profiles – Green

Consensus Standard Data Format v1 API	Required	Conditional
Notifications Get	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	
Notifications Subscription	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	
Notifications - Publishing	Client authentication TLS per [IETF RFC 5246] **OR** private_key_jwt per [IETF RFC 7521] and [IETF RFC 7523].	
Meta	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	
Core	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	FDX Message Encryption FDX Step-up Authentication
Customer	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	FDX Message Encryption FDX Step-up Authentication
Money Movement	OAuth 2.0 - Client credentials grant [IETF RFC 6749]	FDX Message Encryption FDX Step-up Authentication

Part 1B: FDX Consensus Standard Data Format v1 API Security Profile - Blue

1.3 Overview

1.3.1 Introduction

The *FDX Consensus Standard Data Format v1 API Security Profile - Blue* specifies requirements and protocols of the FDX Consensus Standard Data Format v1 for communicating requests and responses. This profile is based on [OIDF FAPI 2.0] and includes additional requirements as noted.

1.3.2 Applicability

Agreements between implementers **SHALL** govern whether *FDX Consensus Standard Data Format v1 API Security Profile - Blue* **SHALL** apply to their implementations of the FDX Consensus Standard Data Format v1.

1.3.3 Scope

The FDX Consensus Standard Data Format v1 API Security Model is intended to enable covered entities to meet the security and interoperability objectives of the rule by defining standard requirements and protocols for communicating requests and responses.

1.4 Profile

1.4.1 General

Implementers of *FDX Consensus Standard Data Format v1 API Security Profile - Blue* **SHALL** comply with [OIDF FAPI 2.0] dated February 19, 2025 for FDX Consensus Standard Data Format v1 APIs as specified in this section.

1.4.2 Authorization code flow

[OIDF FAPI 2.0] with authorization code flow **SHALL** be used for the following FDX Consensus Standard Data Format v1 Endpoints:

- None

1.4.3 Client credentials grant

[OIDF FAPI 2.0] with OAuth 2.0 [IETF RFC 6749] client credentials grant **SHALL** be used for the following FDX Consensus Standard Data Format v1 Endpoints:

- Core
- Customer
- Money Movement
- Notifications - Get

- Notifications - Subscription
- Meta

1.4.4 Other endpoints

[OIDF FAPI 2.0] **SHALL NOT** be used for the following FDX Consensus Standard Data Format v1 Endpoints. For these endpoints implementers **SHALL**, for client authentication, use client authentication TLS per [IETF RFC 5246] **OR** private_key_jwt per [IETF RFC 7521] and [IETF RFC 7523].

- Notifications - Publishing

1.4.5 Additional requirements

1. If a consent grant is revoked, then the refresh token and access token associated with the consent grant **SHALL** be revoked immediately.
2. Authorization servers **SHALL** verify whether scopes included in the authorization request match the consent request object. If there are any extra scopes included in the authorization request, then the request **SHALL** fail.
3. For each request the resource server **SHALL** verify the consent record associated with the access_token entitles the requestor to perform the action on the target resource. If there are any extra scopes or actions, then the request SHALL fail.
4. Access tokens **SHALL** expire after no more than 900 seconds.

Table 3: Consensus Standard Data Format v1 Endpoint Profiles - Blue

Consensus Standard Data Format v1 API	Required	Conditional
Notifications Get	[OIDF FAPI 2.0] - Client credentials grant	
Notifications Subscription	[OIDF FAPI 2.0] - Client credentials grant	
Notifications - Publishing	Client authentication TLS per [IETF RFC 5246] **OR** private_key_jwt per [IETF RFC 7521] and [IETF RFC 7523].	
Meta	[OIDF FAPI 2.0] - Client credentials grant	
Core	[OIDF FAPI 2.0] - Client credentials grant	FDX Message Encryption FDX Step-up Authentication
Customer	[OIDF FAPI 2.0] - Client credentials grant	FDX Message Encryption FDX Step-up Authentication
Money Movement	[OIDF FAPI 2.0] - Client credentials grant	FDX Message Encryption FDX Step-up Authentication

Part 2: FDX Message Encryption

2.1 Overview

2.1.1 Introduction

The FDX Message Encryption standard is a conditional requirement of the FDX Consensus Standard Data Format v1 for communicating requests and responses. If agreements between implementers specify application level encryption, then these requirements apply.

2.1.2 Applicability

Agreements between implementers of the FDX Consensus Standard Data Format v1 **SHALL** govern whether data **SHALL** be encrypted at the application level, which data **SHALL** be encrypted at the application level and which party **SHALL** be the owner of the private key of an asymmetric key pair.

2.1.3 Scope

Protection of data in transport between data providers and third parties **SHALL** be in scope. Protection of data at-rest **SHALL NOT** be in scope.

2.2 Message Encryption

2.2.1 Payload Encryption

Payload encryption from data provider to third party **SHALL** comply with nested JWT implementation specified in [IETF RFC 7519]. The JWS object **SHALL** be encapsulated in the JWE object.

2.2.2 Field Encryption

Field encryption from data provider to third party **SHALL** comply with nested JWT implementation specified in the [IETF RFC 7519]. The JWS object **SHALL** be encapsulated in the JWE object.

2.3 Key Management

A nested JWT implementation requires two pairs of asymmetric keys and one symmetric key. The symmetric key is used for content encryption. With the RSA encryption algorithm, one asymmetric key pair is for the encryption and decryption of the symmetric content encryption key. With the elliptic curve encryption algorithms, one asymmetric key pair is for key agreement in generating a key encryption key. For both the RSA encryption algorithm and the elliptic curve encryption algorithms, the second asymmetric key pair is for signing the payload and validating the signature.

2.3.1 Signing public key and private key pair

The signing asymmetric key pair is for signing the payload in the JWS and validating the signature.

2.3.1.1 Certification Authority

The issuing Certification Authority (CA) **SHALL** be suitable to the implementers. The CA **SHALL** comply with the current approved versions of [NIST 800-57].

2.3.1.2 Exchange

The signing public key **SHALL** be sent in the header of the JWT as specified in [IETF RFC 7515].

Other methods for exchanging public keys, such as through JSON web key set endpoints as specified in [IETF RFC 7517], **SHALL** be determined by the implementers.

2.3.1.3 Key Storage

The owner of the signing private key **SHALL** protect its private key in a system that has been validated as meeting at least FIPS 140-2 level 3 or Common Criteria Protection Profile evaluation assurance level 4.

2.3.1.4 Key Usage

The signing public key - private key pair **SHALL** be used for signing the payload in the JWS and validating the signature.

The signing public key - private key pair **SHALL NOT** be used for other purposes.

The owners of signing private keys **SHALL NOT** share the private keys with parties outside of their organization.

2.3.1.5 Crypto-period

The duration of the crypto-periods for signing public key and private key pairs **SHALL NOT** be longer than two years.

2.3.1.6 Algorithm and key size

Algorithms and key sizes **SHALL** comply with the recommended implementation requirements of the current version of [IANA JOSE].

2.3.2 Key agreement and encryption private and public keys and public key certificates

2.3.2.1 Certification Authority

The issuing Certification Authority (CA) **SHALL** be suitable to the implementers. The CA **SHOULD** comply with the current approved versions of [NIST 800-57].

2.3.2.2 Exchange

The public keys **SHALL** be sent in the header of the JWT as specified in [IETF RFC 7516].

Other methods for exchanging public keys, such as through JSON web key set endpoints as specified in [IETF RFC 7517], **SHALL** be determined by the implementers.

2.3.2.3 Key Storage

The owner of the key agreement private key **SHALL** protect its private key in a system that has been validated as meeting at least FIPS 140-2 level 3 or Common Criteria Protection Profile evaluation assurance level 4.

2.3.2.4 Key Usage

The key agreement public key - private key pair **SHALL** be used for key agreement.

The key agreement public key - private key pair **SHALL NOT** be used for other purposes.

The encryption public key - private key pair **SHALL** be used for encryption.

The encryption public key - private key pair **SHALL NOT** be used for other purposes.

The owners of private keys **SHALL NOT** share the private keys with parties outside of their organization.

2.3.2.5 Crypto-period

The duration of the crypto-periods for signing public key and private key pairs **SHALL NOT** be longer than two years.

2.3.2.6 Algorithm and key size

Algorithms and key sizes **SHALL** comply with the recommended JOSE implementation requirements of the current version of [IANA JOSE].

2.3.2.7 Other key management practices

Other key management practices of the owner of private keys, including but not limited to auditing, key destruction, key recovery and key compromise, **SHALL** comply with the current versions of [NIST 800-57-1] and [NIST 800-57-2].

2.4 Examples

The examples below are provided for clarity only. The individual JWE and JWS RFCs [IETF RFC 7516] and [IETF RFC 7515] referenced supersede implementation details where the examples are inconsistent.

2.4.1 Example: Payload Encryption using a Nested JWT

HTTP Request

```
GET /customers/current HTTP/1.1
...
```

HTTP Response

```
HTTP/1.1 200 OK
Content-type: application/jwt
...
//Nested JWT (JWS encapsulated in a JWE)
//JWE outside, JWS inside
{
    "protected": BASE64({
        "alg":"RSA-OAEP",
        "enc":"A256GCM"
    }),
    "unprotected:": {
        "jku":"https://server.example.com/keys.jwks"
    },
    "ciphertext": BASE64(ENCRYPT<A256GCM using
'encrypted_key'>({ //JWS to be encrypted (shown before
encryption below)
        "protected": {
            "kid":"123",
            "jku":"https://server.example.com/keys.jwks",
            "alg":"RS256"
        },
        "payload": BASE64({//The encrypted content will be valid
schema of the `/customers/current` endpoint ie. `Customer`
entity
            "customerId" : "abc-12345-xyz"
            "name" : {
                "first" : "John",
                "last" : "Smith"
            }
            "email" : ["jsmith@email.com"]
        }),
        "signature": //Using Data Provider signing private key

"nupgm7iFqSnERq9GxszwBrsYrYfMuSfUGj8tGQlkY3Ksh3o_IDfq1GO5ngHQLZu
YPD-
8qPIovPBEVomGZCo_jYvsbjmYkalAStmF01TvSoXQgJd09ygZstH0liKsmINSti"
    ))},
    "iv": "AxY8DCtDaGlsbGljb3RoZQ"
    "encrypted_key": //A256GCM key encrypted using RSA-OAEP with
Data Access Platform's Public Key
```

```
"UGhIOguC7IuEvf_NPVaXsGMoLOmwvc1GyqlIKOK1nN94nHPoltGRhWhw7Zx0-
kFm1NJn8LE9XShH59_i8J0PH5ZZyNfGy2xGdULU7sHNF6Gp2vPLgNZ__deLKxGHZ
7PcHALUzoOegEI-8E66jX2E4zyJKx-YxzZIItRzC5hlRirb6Y5Cl_p-
ko3YvkkysZIFNPccxRU7qve1WYPxqbb2Yw8kZqa2rMWI5ng8OtvzlV7elprCbuPh
cCdZ6XDP0_F8rkXds2vE4X-ncOIM8hAYHHi29NX0mcKiRaD0-D-ljQTP-
cFPgwCp6X-nZZd9OHBv-B3oWh2TbqmScqXMR4gp_A",
    "tag": //Protection of payload
         "Mz-VPPyU4RlcuYv1IwIvzw"
}
```


2.4.2 Example: Field-level Encryption using a Nested JWT

HTTP Request

```
GET /customers/current HTTP/1.1
...
```

HTTP Response

```
HTTP/1.1 200 OK
Content-type: application/json
...
{
    "customerId" : "abc-12345-xyz"
    "encryptedName" : //Nested JWT (JWS encapsulated in a JWE).
JWE outside, JWS inside
    {
        "protected": BASE64{
            "alg":"RSA-OAEP",
            "enc":"A256GCM"
        }),
        "unprotected": {
            "jku":"https://server.example.com/keys.jwks"
        }
        "aad": BASE64({ //Additional Authenticated Data that
contains the JWS with non-PII
            "protected": {
                "kid":"123",
                "jku":"https://server.example.com/keys.jwks",
                "alg":"RS256"
            },
            "payload": BASE64({ //Non-PII fields that are signed
but not encrypted. For example only.
                "bank": "Chase"
            }),
```

```
            "signature": //Signature provides a means to verify
the authenticity
                "DtEhU3ljbEg8L38VWAfUAqOyKAM6-Xx-
F4GawxaepmXFCgfTjDxw5djxLa8ISlSApmWQxfKTUJqPP3-Kg6NU1Q"
                 }),
        "ciphertext": BASE64(ENCRYPT<A256GCM using
'encrypted_key'>({ //JWS to be encrypted (shown before
encryption below)
            "protected": {
                "kid":"123",
                "jku":"https://server.example.com/keys.jwks",
                "alg":"RS256"
            },
            "payload": BASE64({ //Encrypted content matches the
schema of the "name" field of the `Customer` entity. The
`CustomerName` entity
                "first" : "John",
                "last" : "Smith"
            }),
            "signature":

"nupgm7iFqSnERq9GxszwBrsYrYfMuSfUGj8tGQlkY3Ksh3o_IDfq1GO5ngHQLZu
YPD-
8qPIovPBEVomGZCo_jYvsbjmYkalAStmF01TvSoXQgJd09ygZstH0liKsmINSti"
        }),
        "iv": "AxY8DCtDaGlsbGljb3RoZQ",
        "encrypted_key": //A256GCM key encrypted using RSA-OAEP
with Data Access Platform's Public Key

"UGhIOguC7IuEvf_NPVaXsGMoLOmwvc1GyqlIKOK1nN94nHPoltGRhWhw7Zx0-
kFm1NJn8LE9XShH59_i8J0PH5ZZyNfGy2xGdULU7sHNF6Gp2vPLgNZ__deLKxGHZ
7PcHALUzoOegEI-8E66jX2E4zyJKx-YxzZIItRzC5hlRirb6Y5Cl_p-
ko3YvkkysZIFNPccxRU7qve1WYPxqbb2Yw8kZqa2rMWI5ng8OtvzlV7elprCbuPh
cCdZ6XDP0_F8rkXds2vE4X-ncOIM8hAYHHi29NX0mcKiRaD0-D-ljQTP-
cFPgwCp6X-nZZd9OHBv-B3oWh2TbqmScqXMR4gp_A",
        "tag": //Protection of AAD and payload
            "Mz-VPPyU4RlcuYv1IwIvzw"
    }
}
```

Part 3: FDX Step-up Authentication

3.1 Overview

3.1.1.1 Introduction

For certain sensitive operations, data providers may need to require elevated authentication strengths or updated recentness. This standard specifies a method for a data provider to communicate to a third party that an access token does not meet its requirements for a certain operation and how these requirements can be met.

3.1.1.2 Scope

This standard applies to all FDX endpoints.

User experience is out of scope of this standard.

3.1.1.3 Applicability

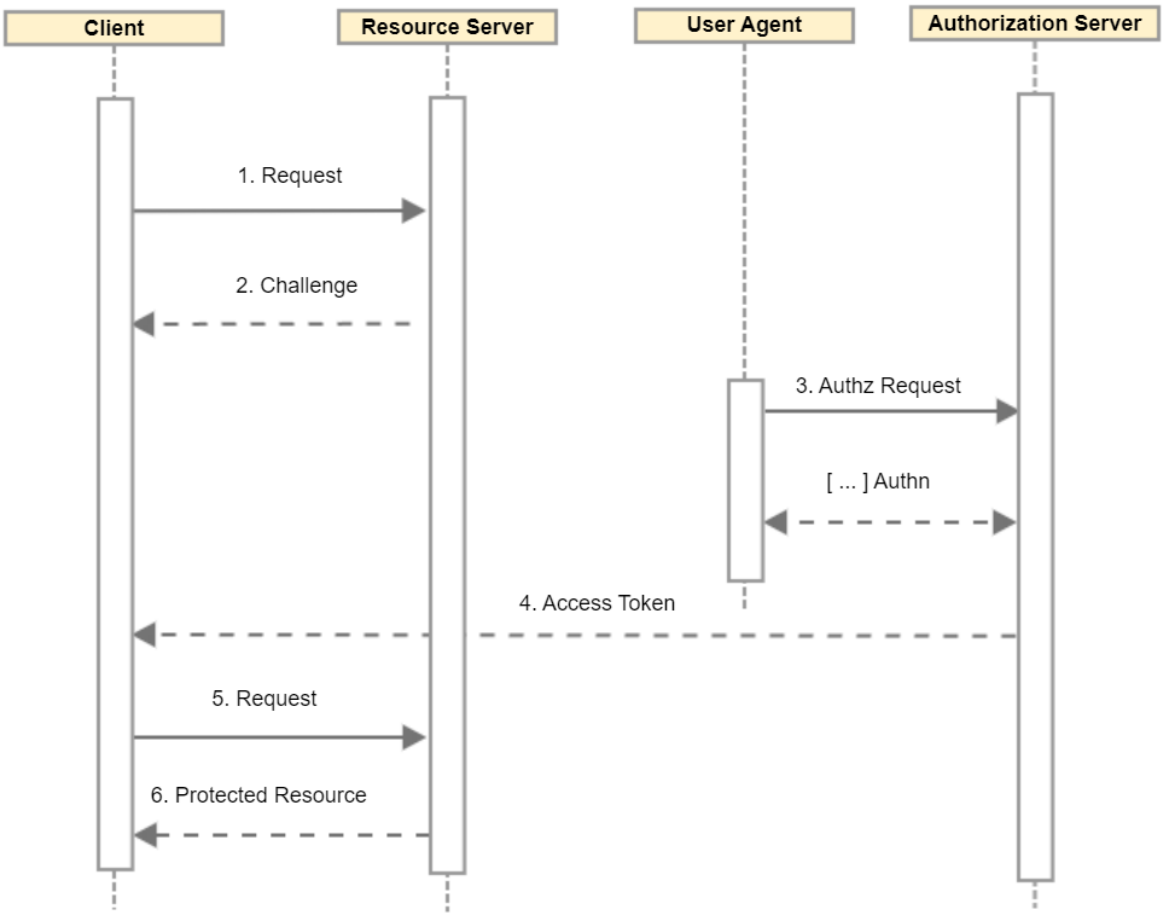
The FDX Step-up Authentication standard is a conditional requirement of the FDX Consensus Standard Data Format v1 for communicating requests and responses. If agreements between implementers of the FDX Consensus Standard Data Format v1 specify step-up authentication, then these requirements apply.

3.2 OAuth 2.0 Step-up Authentication Challenge Protocol

1. FDX **SHALL** adopt [IETF RFC 9470] as the method for implementing step-up authentication for applicable implementations.

3.3 Implementer's Notes

1. Access token caching strategy is not specified in this standard. The implementers **SHALL** determine the access token caching strategy. The term *token caching strategy* is defined in Terms and Definitions in the Introduction.
2. Sequence Diagram



1. Client presents access token and requests protected resource.
2. Resource server determines access token has insufficient authentication or recentness. Resource server denies request. Resource server returns challenge.
3. Client directs user agent to the authorization server with the appropriate acr values or max age.
4. [...] Required authentication of end user.
5. Authorization server returns a new access token to the client.
6. Client presents new access token and repeats request for protected resource.
7. Resource server verifies access token and returns requested protected resource.

Normative References

[FDX CSDF] FDX Consensus Data Format

[FIPS140-2] FIPS PUB 140-2 Security Requirements for Cryptographic Modules. May 2001.

[IETF RFC 2119] Key words for use in RFCs to Indicate Requirement Levels. March 1997.

[IETF RFC 5246] The Transport Layer Security (TLS) Protocol Version 1.2. August 2008.

[IETF RFC 6749] The OAuth 2.0 Authorization Framework 2.0. October 2012.

[IETF RFC 7515] JSON Web Signature (JWS). May 2015.

[IETF RFC 7516] JSON Web Encryption (JWE). May 2015.

[IETF RFC 7517] JSON Web Key (JWK). May 2015.

[IETF RFC 7518] JSON Web Algorithms (JWA). May 2015.

[IETF RFC 7521] Assertion Framework for OAuth 2.0 Client Authentication and Authorization Grants. May 2015.

[IETF RFC 7523] JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants. May 2015.

[IETF RFC 9470] OAuth 2.0 Step-up Authentication Challenge Protocol. September 2023.

[IETF RFC 9700] Best Current Practice for OAuth 2.0 Security. January 30, 2025.

[IETF JOSE] JSON Object Signing and Encryption (JOSE).

[NIAP Common Criteria] Common Criteria Protection Profile for Application Software. November 5, 2014¬.

[NIST 800-57-1] NIST Special Publication 800-57 Part 1 Revision 5: Recommendation for Key Management. May 2020.

[NIST 800-57-2]. NIST Special Publication 800-57 Part 2 Revision 1: Recommendation for Key Management. May 2019.

[OIDF FAPI] OIDF FAPI 2.0. February 19, 2025.

[RR PFDR] CFPB Required Rulemaking on Personal Financial Data Rights. January 17, 2025.

Informative References

[FDX RFC 0011] Message Encryption. (Replaced and made obsolete by this standard)

[FDX RFC 0223] Control Considerations for Step-up Authentication. (Replaced and made obsolete by this standard)